

# Semaine méthodes

## Matière

- Les méthodes

## Objectifs

- Comprendre les différents objectifs visés en introduisant des méthodes
- Comprendre le principe du passage de paramètres et de l'appel de méthode
- Pouvoir écrire un if imbriqué dans une méthode
- Comprendre le rôle de la *JavaDoc*

**Pour les exercices de cette semaine, créez un nouveau projet IntelliJ et appelez-le **ALGO\_methodes**.**

## Exercices obligatoires

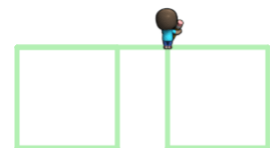
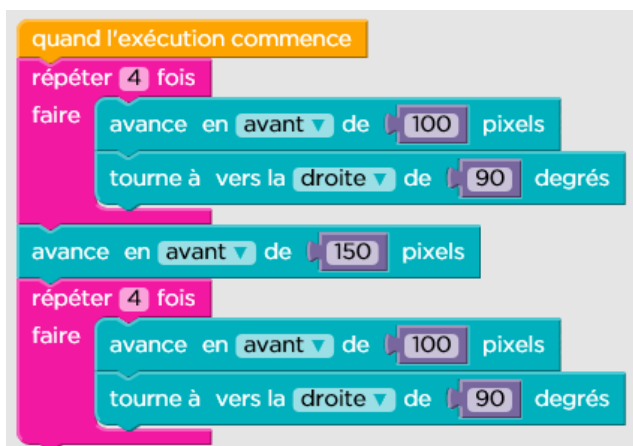
### A La tortue : le retour !

**DRY (Don't repeat yourself) - Ne vous répétez pas !**

La factorisation de code consiste à rassembler les suites d'instructions identiques en une « méthode » pour améliorer la lisibilité et en faciliter la correction et les modifications ultérieures.

Ce sont toujours les mêmes formes géométriques qui apparaissent dans les dessins que vous avez programmés avec la tortue : des carrés, des triangles, des cercles, ...

Voilà une belle impression de « copier-coller » de mêmes suites d'instructions au sein d'une même classe et aussi d'une classe à l'autre !



Donnons à la tortue plus de responsabilités :

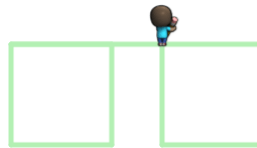
```
tortue.  
m dessinerUnCarre(double longueur)  
m dessinerUnTriangle(double longueur)  
m accelerer()  
m avancer(double pixel)  
m tournerADroite(int degre)  
m definirCouleur(String couleur)  
m tournerAGauche(int degre)
```

A1 La classe *Tortue* fournie cette semaine contient déjà la méthode `dessinerUnCarre(double longueur)`.

Ajoutez-lui la méthode `dessinerUnTriangle(double longueur)`.

En utilisant les méthodes `dessinerUnCarre()` et `dessinerUnTriangle()` qui ont été ajoutées dans la classe *Tortue* :

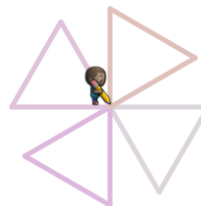
A2.1 Ecrivez la classe *DessinLunettes*



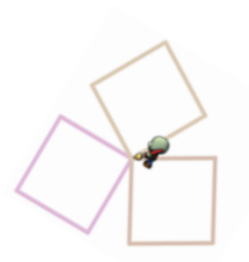
A2.2 Ecrivez la classe *DessinEnveloppe* qui dessine une enveloppe.



A2.3 Ecrivez la classe *DessinFleur*



A2.4 Ecrivez la classe *DessinVentilateur*.



A2.5 Ecrivez la classe *DessinCarresEmboités*.



## **B La classe *Utilitaires***



Technique Bottom-up : on commence par réaliser toute une série d'outils qui pourront servir pour élaborer différents programmes.

Nous ne comptons plus le nombre de fois qu'on a eu besoin de copier-coller la méthode `unEntierAuHasardEntre()`.

Jusqu'à présent, nous acceptons les entrées clavier sans jamais les vérifier. Que se passe-t-il si on encode un prix négatif ? Une cote supérieure à 20 ? Un diviseur égal à 0 ?

Nous avons décidé de rassembler dans une même classe des **méthodes utiles à de nombreuses classes**. Nous avons appelé cette classe ***Utilitaires***.

Au fur et à mesure de l'année, il serait intéressant de l'avoir sous la main et de penser à la compléter.

Cette classe propose des méthodes **statiques**.

Exemple d'utilisation :

```
int de = Utilitaires.unEntierAuHasardEntre(1, 6);
```

B1 Complétez les méthodes `lireReelComprisEntre()` et `lireOouN()` de la classe *Utilitaires*. Améliorez la classe ***CalculMoyenne*** en validant toutes les lectures clavier.

B2 Revoyez la classe ***DivisionEntiere***. Il faut éviter les divisions par 0 ! Commencez par ajouter la méthode `lireEntierNonNul()` dans la classe *Utilitaires* et utilisez-là. N'oubliez pas d'ajouter la *JavaDoc*.

B3 Revoyez la classe *Statistiques*. On voudrait que toutes les lectures clavier soient validées. Commencez par compléter la classe *Utilitaires* en y ajoutant des méthodes `lire...()`.

B4 Dans la classe *Utilitaires*, ajoutez la méthode :

```
int max3(int entier1, int entier2, int entier3)
```

Revoyez la classe *RechercheMax3*. Elle doit appeler la méthode `max3()`.

B5

Le *return* provoque la sortie de la méthode.

Il ne faut plus faire apparaître les *else* dans les *ifs* imbriqués.

**ATTENTION :**

Ce n'est pas parce qu'on ne voit plus de *else* que les *ifs* ne sont pas imbriqués. C'est juste que le *else* n'est plus obligatoire.

Supprimez les *else* qui ne sont pas obligatoires dans votre méthode `max3()`.

Comparez votre solution de votre méthode `max3()` avec celle-ci :

```
public int max3(int entier1, int entier2, int entier3){
    if(entier1 > entier2 && entier1 > entier3)
        return entier1;
    //Ce n'est pas entier1
    if(entier2 > entier3)
        return entier2;
    //Ce n'est pas entier1, ce n'est pas entier2, c'est forcément entier3
    return entier3;
}
```

B6 Dans la classe *Utilitaires*, ajoutez la méthode :

```
int lancerDe()
```

Cette méthode doit faire appel à la méthode `unEntierAuHasardEntre()` !

Ce n'est pas que la méthode `main()` qui peut faire appel à des méthodes.

Revoyez la classe *CombatAMort*.

Supprimez-y la méthode `unEntierAuHasardEntre()`. Elle va appeler la méthode `lancerDe()` de la classe *Utilitaires*.

## C BMI

L'indice de masse corporelle (BMI), utilisé par les diététiciens, est révélateur de l'état d'une personne.

Une personne normale devrait avoir un indice compris entre 20 et 25. En dessous de 20 elle est considérée comme mince, au-dessus de 25 elle est en embonpoint. Au-delà de 30 elle est obèse.

Voici comment est calculé cet indice :  $BMI = P/T^2$

T est la taille de la personne exprimée en mètre et P le poids de la personne exprimé en kg.

Le programme lit la taille (réel) et le poids (entier) de la personne. Il calcule et affiche son BMI et annonce son état.

En semaine 2 (semaine if) : Vous avez écrit la classe *CalculBMI*.

Vous allez écrire 2 autres variantes de cette classe.

C1 classe *calculBMIV2*



Technique Top-down : on pense d'abord au programme principal en laissant pour plus tard l'étude des détails, que l'on confiera à des méthodes

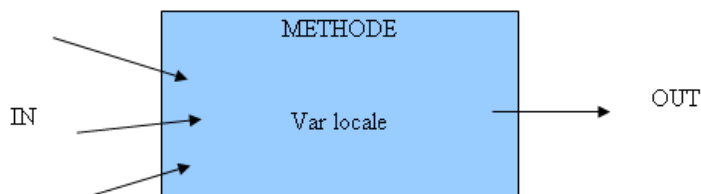
On exprime, en français structuré, les principales étapes du programme (**macro-code**) :

« On » lit les données du patient.

« On » calcule le bmi

« On » évalue son état.

On procède à une découpe en méthodes :



Dans la classe *CalculBMIV2*, complétez les méthodes **statiques** :

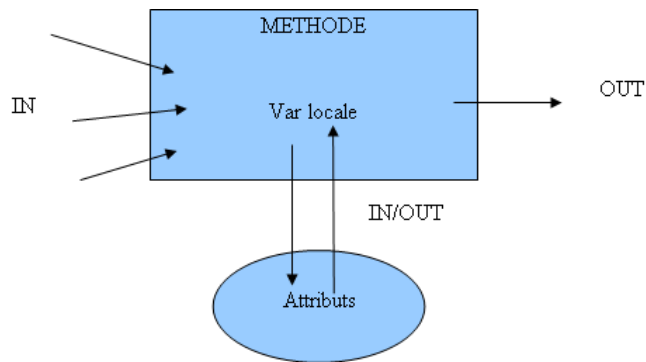
```
double calculBmi(double poids, double taille)
String donnerEtat(double bmi)
```

C2 classe *calculBMIV3*



Technique Bottom-up : on commence par réaliser toute une série d'outils qui pourront servir pour élaborer différents programmes.

On commence par écrire une classe *Patient*.



AP00

La classe *calculBMIV3* :

« On » lit les données du patient et « on » crée un objet de la classe *Patient*  
 « On » utilise les méthodes qui nous intéressent de la classe *Patient*.

Dans la classe *Patient*, complétez les méthodes (non statiques) :

```
double calculBmi()
String donnerEtat()
```

Testez la classe *CalculBMIV3*.

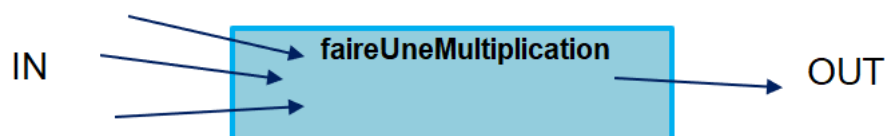
## D Les classes *Multiplication1*, *Multiplication2*, ...



Plusieurs programmes « *Multiplication* » sont demandés.

Ils sont différents mais possèdent la même caractéristique : proposer **plusieurs** exercices de multiplication !

On « imagine » facilement pour chacun de ces programmes introduire une méthode qui s'appelle `faireUneMultiplication()`. Cette méthode, comme son nom l'indique, s'occupe d'une multiplication. La méthode `main()` contiendra la répétitive dans laquelle se fera l'appel à cette méthode.



## D1 classe *Multiplication1*

Cette classe propose 5 multiplications de 2 nombres entiers.

La valeur minimale des 2 nombres à multiplier est 0.

C'est l'utilisateur qui choisit la valeur maximale de ces 2 nombres. Cette valeur maximale sera la même pour toutes les multiplications.

Pour chaque multiplication, l'utilisateur n'a droit qu'à un essai. Si la réponse entrée au clavier n'est pas bonne, il faut afficher la bonne réponse.

A la fin, le programme affiche le nombre de multiplications réussies.

Complétez la classe fournie *Multiplication1*.

Suivez bien la JavaDoc de la méthode `faireUneMultiplication()`.

## D2 classe *Multiplication2*

Cette classe propose des multiplications de 2 nombres entiers.

La valeur minimale des nombres à multiplier est 0 et la valeur maximale est 10.

Pour chaque multiplication, tant que l'utilisateur n'a pas donné la bonne réponse, il devra la recommencer

Le programme termine lorsque l'utilisateur aura réussi 3 multiplications du premier coup.

Complétez la classe fournie *Multiplication1*.

Suivez bien la JavaDoc de la méthode `faireUneMultiplication()`.

## Exercices supplémentaires



B5 Dans la classe *Utilitaires*, ajoutez la méthode :

```
int max2(int entier1, int entier2)
```

Réécrivez la méthode `max3()` en utilisant la méthode `max2()`.

Dans la méthode `max3()`, on ne veut pas voir d'alternative (**pas de if !**).

## D3 classe *Multiplication3*

Cette classe propose 5 multiplications de 2 nombres entiers.

Au départ, la valeur minimale des nombres à multiplier est 0 et la valeur maximale est 5.

Ces 2 bornes augmenteront de 5 à chaque multiplication réussie.

Pour chaque multiplication, l'utilisateur n'a droit qu'à un essai. Si la réponse entrée au clavier n'est pas bonne, il faut afficher la bonne réponse.

Complétez la classe *Multiplication3*.

#### D4 classe *Multiplication4*

Cette classe propose des multiplications de 2 nombres entiers.

La valeur minimale des nombres à multiplier est 1 et la valeur maximale est 10.

L'utilisateur choisit le nombre d'essais auquel il a droit.

Ce nombre sera la même pour toutes les multiplications.

Le programme donne la réponse si l'utilisateur ne l'a pas trouvée au terme du nombre d'essais accordé.

Après chaque multiplication, le programme demande à l'utilisateur s'il en veut une autre.

L'utilisateur doit répondre par le caractère 'Y' pour en avoir une autre.

Complétez la classe *Multiplication4*.

### Exercice défi



B6 Dans la classe *Utilitaires*, ajoutez la méthode :

```
char lireCharPermis(String caracteresPermis)
```

Cette méthode demande à l'utilisateur d'entrer un caractère au clavier. Ce caractère devra être contenu dans la chaîne de caractères passée en paramètre à la méthode. Si l'utilisateur se trompe, on lui repose la question jusqu'à avoir un caractère admis. La méthode renverra le caractère entré par l'utilisateur.

Utilisez cette méthode dans la classe *NombreMystereInverse* écrite lors de la semaine *while*.